

Incentivizing Agentic Reasoning Capability with Outcome Supervision for KBQA

Zhuo Chen
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
chenzhuo23s@ict.ac.cn

Fei Wang*
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
wangfei@ict.ac.cn

Zixuan Li*
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
lizixuan@ict.ac.cn

Zhao Zhang
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
zhangzhao2021@ict.ac.cn

Weiwei Ding
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
wwdingchn@163.com

Chuangang Yang
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
yanghuangang@ict.ac.cn

Yongjun Xu*
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
xuyongjun@ict.ac.cn

Xiaolong Jin
State Key Laboratory of AI
Safety, Institute of
Computing Technology,
Chinese Academy of
Sciences
University of Chinese
Academy of Sciences
Beijing, China
jinxiaolong@ict.ac.cn

Abstract

Knowledge Base Question Answering (KBQA) aims to answer natural-language questions over a structured Knowledge Base (KB). Recent work improves KBQA by adopting an agentic reasoning paradigm, in which Large Language Models (LLMs) iteratively decompose a question, generate its corresponding logical queries, and interact with the KB to derive the answer. However, these methods typically fine-tune LLMs on reasoning trajectories synthesized via process supervision, which offers weak incentives for exploration and thus fails to strengthen the agentic reasoning ability. In this paper, we propose AGENTICKBQA, an LLM that can autonomously perform agentic reasoning on KBs to obtain answers. To incentivize autonomous exploration, it trains the LLM under outcome-only supervision via a multi-stage curriculum reinforcement learning with an easy-to-hard curriculum. To establish foundational agentic capabilities, it first fine-tunes the LLM on a small set of high-quality trajectories obtained through outcome-based rejection sampling. Then, to alleviate the reward sparsity inherent in outcome-only supervision, it applies multi-stage curriculum RL with reward schedules that progress from easy to hard. Trained with outcome-only supervision, AGENTICKBQA exhibits

powerful reasoning behaviors and consistently outperforms prior approaches across three mainstream datasets. Notably, on the zero-shot subset of GrailQA, AGENTICKBQA achieves up to an 11.1% relative improvement while using only one-twelfth of the training data, demonstrating strong agentic reasoning capabilities. Code is available at <https://github.com/ICT-GoKnow/KnowCoder-A1>.

CCS Concepts

• Information systems → Question answering.

Keywords

Knowledge Base Question Answering; Large Language Model; Knowledge Graph

ACM Reference Format:

Zhuo Chen, Fei Wang, Zixuan Li, Zhao Zhang, Weiwei Ding, Chuangang Yang, Yongjun Xu, and Xiaolong Jin. 2026. Incentivizing Agentic Reasoning Capability with Outcome Supervision for KBQA. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792662>

1 Introduction

Knowledge Base Question Answering (KBQA) leverages the rich semantic information in Knowledge Bases (KB) to deeply understand natural language questions and provide precise answers. Since KBQA holds significant value in applications such as search engines, intelligent healthcare, and financial risk control [2, 14, 33, 39], it has garnered extensive attention in recent years. Although KBQA performs well on simple questions, it still struggles with complex

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792662>

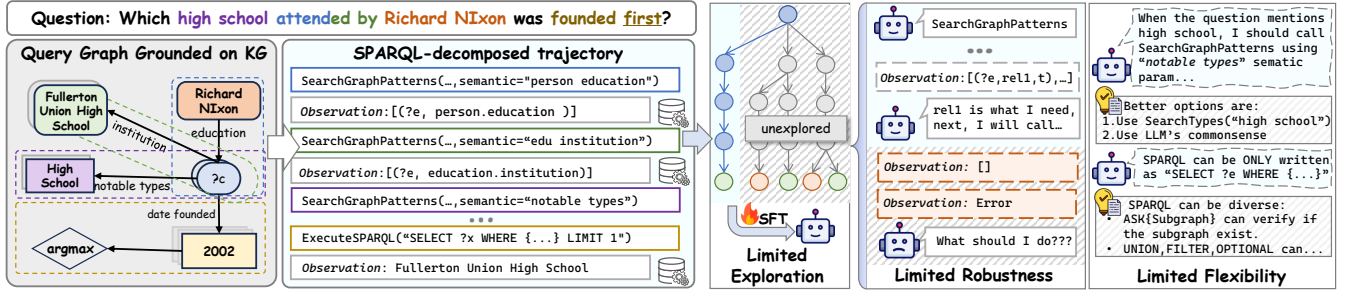


Figure 1: The key limitations of existing agentic approaches that rely on process supervision.

questions in real-world scenarios. This difficulty primarily arises from generalizing to a wide range of unseen complex questions and adapting to the heterogeneous structures of KBs.

Existing semantic-parsing-based [16, 32] and information-retrieval-based KBQA approaches [20, 34] rely on predefined workflows (e.g., generate-then-retrieve or retrieve-then-rank pipelines), resulting in error propagation and adaptability limitations. Recently, a series of approaches has adopted an agentic reasoning paradigm. As illustrated in the middle of Fig. 1, under this paradigm, LLMs autonomously perform task planning, tool calling, and ultimately derive precise answers through multi-turn interactions with KBs. Specifically, existing agentic approaches fall into two categories, i.e., prompting-based [35, 40] and finetuning-based approaches [9, 15, 47]. The former uses hand-crafted few-shot prompts to guide reasoning, relying on the intrinsic reasoning ability of LLMs. However, few examples are inadequate for the model to understand complex questions, perceiving the KB structure, resulting in low performance. In response, finetuning-based approaches adapt open-source LLMs by constructing agentic reasoning trajectories and fine-tuning the model to follow them. However, most of them rely on process supervision. Since manually creating high-quality trajectories is labor-intensive, they typically synthesize reasoning trajectories by decomposing gold logical forms (e.g., SPARQL). Besides, they learn from trajectories via Supervised Fine-Tuning (SFT) to employ the model with agentic reasoning ability.

Despite the progress, existing finetuning-based approaches rely heavily on process supervision both in data generation and model training. They encourage LLMs to exploit idea trajectories rather than autonomously exploring alternative ones. Consequently, they have two critical limitations:

- **Limited Robustness.** Under process supervision, as shown in the left part of Fig. 1, training trajectories are typically decomposed from gold logical forms, with each step being idealized and error-free. However, such trajectories omit the noise and uncertainty inherent in real-world interactions. As a result, models always struggle when exposed to the noisy, such as tool-call failures and empty results, as they are trained on ideal trajectories.
- **Limited Flexibility.** For many questions, there exists more than one correct trajectory, and in some cases, certain trajectories may even outperform the gold logical forms. Training on trajectories only derived from gold logical forms constrains the model to low-diversity behavioral patterns. As shown in Fig. 1, when the model is presented with an example question containing a “high

school” type constraint, it insists on finding a predicate with the meaning “notable type” and applies the type constraint. However, more efficient solutions exist, such as leveraging common sense to identify entities of a specific type from the candidate set, or using “SearchTypes” to more directly constrain the entity types.

Motivated by this, we propose AGENTICKBQA, a framework that enables an LLM to autonomously performing task decomposition, invoking tools, refining formal queries, and executing them to obtain answers. To incentivize autonomous exploration, AGENTICKBQA adopts a multi-stage curriculum reinforcement learning approach that relies mainly on outcome supervision. First, to break free from the dependency on process annotation, it employs an outcome-based rejection sampling strategy to curate high-quality trajectories and perform to endow the model with a foundational exploration capability. To mitigate the reward sparsity induced by outcome-only signals, AGENTICKBQA employs multi-stage curriculum RL on reward schedules that progress from easy to hard, steadily strengthening autonomous exploration and complex reasoning. Trained with only around 6700 outcome-supervised samples, AGENTICKBQA consistently outperforms previous agentic KBQA approaches across three mainstream datasets, i.e., WebQSP, CWQ, and GrailQA. Using 12× less training data, it achieves an F1 score of 80.5% on GrailQA, achieving a 3.3% relative improvement over KBQA-o1, the previous SOTA agentic-based approach. Notably, on the zero-shot subset of GrailQA, AGENTICKBQA achieves a relative improvement of up to 11.1% compared to KBQA-o1, demonstrating the strong generalization ability of the proposed approach.

In summary, our contributions are as follows:

- We propose AGENTICKBQA, our first agentic reasoning model for KBQA. Relying mainly on outcome supervision, it learns to act as a robust and flexible agent, capable of recovering from errors and strategically exploring diverse reasoning trajectories.
- To address the reward sparsity inherent in outcome-only supervision, we propose a multi-stage curriculum reinforcement learning with an easy-to-hard curriculum, progressing from easy to hard reward criteria while discouraging reward hacking.
- Extensive experiments on three mainstream datasets demonstrate that AGENTICKBQA consistently outperforms previous agentic KBQA approaches, particularly in challenging zero-shot scenarios.

2 Related Work

2.1 Traditional KBQA Methods

Traditional KBQA methods can be categorized into two primary kinds of approaches: Information Retrieval-based (IR-based) and Semantic Parsing-based (SP-based) ones. IR-based methods [20, 24, 28, 31, 34, 44] typically retrieve query-relevant information from the KB and subsequently rank candidate answers. To better capture inter-relationships between questions and the KB, some studies leverage memory-based architectures [1, 3] to facilitate complex reasoning, or perform a path-walking process over the KB to derive the answer [26, 27]. Conversely, SP-based methods [4, 5, 16, 17, 32, 36, 43] transform the natural language question into a logical query, link the entities and relations, and execute the completed query to obtain the answer. With the advent of LLMs, they have been employed to enhance both approaches, serving as powerful subgraph retrievers [10, 21, 45] or semantic parsers [18, 22, 25]. However, these enhancements do not fundamentally alter the predefined, multi-stage workflow, which brings inflexibility and error propagation.

2.2 Agentic KBQA Methods

Recently, a new paradigm of agentic reasoning has emerged, which views the LLM as an agent that uses tools to interact with the KB, progressively reason, and finally find the answer. They can be divided into two main categories: prompt-based and finetuning-based ones. The former methods, such as Interactive-KBQA [40], QueryAgent [13], Readi [7], and TOG [35], leverage carefully designed interaction strategies and prompts to elicit the model’s reasoning capabilities. However, these methods struggle to equip the model with sufficient agentic capability to effectively perceive the KB and execute complex reasoning with the limited number of examples. Finetuning-based methods aim to distill reasoning abilities into smaller, open-source LLMs. Approaches [9, 11, 15, 47] achieve this by constructing high-quality reasoning trajectories and finetuning on them. Despite their success, these approaches typically rely on process-supervised SFT over trajectories derived from augmented gold programs. This reliance restricts the model’s capacity for autonomous exploration, leading to poor robustness in the face of diverse reasoning paths. Some recent works have used Monte Carlo Tree Search (MCTS) methods [23, 41] to expand samples during inference for incremental fine-tuning, but this is an indirect way to foster exploration and still operates within the confines of the process-supervised SFT paradigm.

3 Preliminaries

Knowledge Base. A Knowledge Base (KB) is a structured knowledge graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{F})$, where $\mathcal{E}$ is the set of entities, $\mathcal{R}$ is the set of relations, and $\mathcal{F}$ is the set of factual triples. Each triple $f \in \mathcal{F}$ takes the form (h, r, t) , where $h, t \in \mathcal{E}$ are the head and tail entities, and $r \in \mathcal{R}$ is the relation.

KBQA. Given a natural-language question q , the KB $\mathcal{G}$, and a set of topic entities $E_q \subseteq \mathcal{E}$ mentioned in q , the objective of the KBQA task is to find the answer set that maximizes the conditional probability $P(\mathcal{A}_q|q, E_q, \mathcal{G})$. Notably, following prior work [23], we

assume entity mentions in q are linked to the knowledge graph $\mathcal{G}$ and given as an input.

Agentic Reasoning Paradigm for the KBQA Task. In the agentic KBQA paradigm, the LLM is viewed as an agent learning a policy π to interact with the environment. At each step t , the agent chooses an action $a_t \in \text{ACTION}$ based on the question q and its historical reasoning trajectory $\tau_t = \{(c_1, a_1, o_1), \dots, (c_t, a_t, o_t)\}$, where ACTION denotes the action space, c_t denotes the explicit thinking process before the action, and o_t is the observation from the environment. This interactive process continues until the agent finds the final answer or reaches the maximum number of steps.

4 Methodology

4.1 Agentic Initialization

AGENTICKBQA follows a ReAct-style [42] agent paradigm that interleaves explicit reasoning and tool use over a KB environment, and terminates with a final answer.

Environment. We view the KB and the tool executor as the environment: $\mathcal{M} = (\mathcal{G}, \text{Exec})$, where Exec executes tool calls against $\mathcal{G}$ and returns structured observations. It is worth noting that error messages are retained to provide corrective signals to help LLM refine subsequent tool calls.

Action Space. At each step, the agent first produces an explicit thinking process and then makes a tool call from a finite action set ACTION. We instantiate three tools based on Interactive-KBQA [40]:

- **SEARCHTYPES(typeName):** Given a type name, retrieves and ranks candidate types in similarity order.
- **SEARCHGRAPHPATTERNS(sparql, semantic):** Given a partial SPARQL sketch (assumed to start with `SELECT DISTINCT ?x WHERE { . . . }`), returns up to 10 one- and two-hop subgraphs in which the variable `?x` appears as the head or tail object. An optional semantic parameter can be provided to rank the returned subgraphs in similarity order.
- **EXECUTESPARQL(sparql):** Given a SPARQL query, executes it against a sandboxed endpoint over $\mathcal{G}$ and returns the result. This enables flexible query on the graph, such as hypothesis testing (e.g., subgraph existence) or query execution.

State and Trajectory. The trajectory τ also serves as the agent’s state, is the history of all past interactions. The trajectory at step t is denoted as:

$$\tau_t = \{(c_1, a_1, o_1), \dots, (c_t, a_t, o_t)\}. \quad (1)$$

Interactive Process. Given a question q , the agent begins with an initial prompt:

$$\text{Prompt} = (\text{Inst}, q, E_q), \quad (2)$$

where Inst encapsulates the task instructions, tool definitions, and few-shot demonstrations. At each step t , the agent generates its thought and action via the LLM:

$$c_t, a_t = \text{LLM}(\text{Prompt}, \tau_{t-1}). \quad (3)$$

The resulting observation $o_t = \text{Exec}(a_t)$ is then appended to the history to form $\tau_t = \tau_{t-1} \cup (c_t, a_t, o_t)$. This loop continues until the agent generates a final answer set $\mathcal{A}_q$ or reaches the maximum step limit T . Notably, we do not employ complex context management on the trajectory. Instead, we simply restrict tool response lengths,

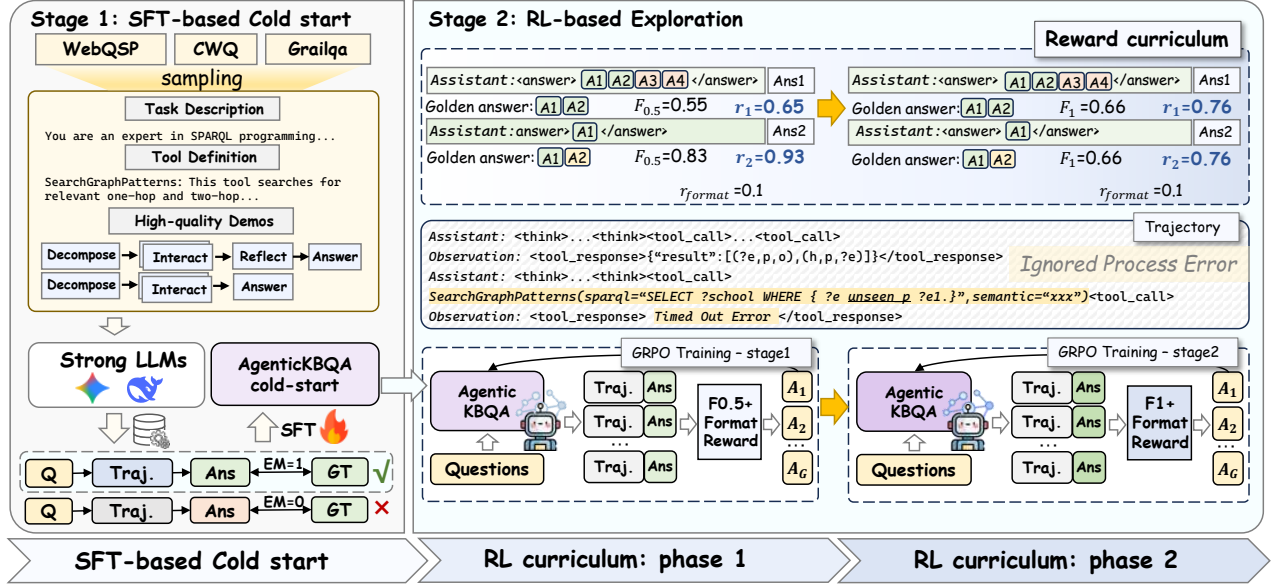


Figure 2: An overview of the training framework of AGENTICKBQA. Stage 1 (left): the SFT-based cold-start process, where high-quality trajectories are curated from strong LLMs to fine-tune an initial agent. Stage 2 (right): the multi-phase Reinforcement Learning curriculum, where the agent is progressively improved through exploration and a dynamic reward strategy.

limiting them to the top-10 results or a maximum of 1,000 characters while retaining the full interaction history.

4.2 SFT-based Cold-start Stage

Due to the heterogeneous structures of KBs and the complexity of questions, employing a base model with insufficient reasoning and tool-use capabilities as an agent makes it difficult to guarantee the discovery of trajectories to the correct answer. To mitigate this issue, we introduce a cold-start stage where the agent is fine-tuned on reasoning trajectories to acquire foundational perception and reasoning capabilities. Instead of relying on program-decomposed trajectories, we generate trajectories via outcome-based rejection sampling. Moreover, to avoid overfitting while ensuring sufficient guidance, we adhere to the principle of constructing a dataset that is concise, yet high in quality and diversity. In the following, we will first describe this data curation process and then detail the model’s training procedure.

4.2.1 Data Construction. We design an outcome-based data construction strategy, the overall pipeline is illustrated in the left panel of Fig. 2, which contains three core components:

First, to ensure the model learns a diverse set of fundamental reasoning patterns, we curate a mixed question set across several sources. From WebQSP, we balance one- and two-hop questions to teach varied reasoning depths. From CWQ, we up-sample infrequent question types to guarantee exposure to complex compositional structures. Finally, from GrailQA, we up-sample questions without topic entities to improve the model’s ability to generalize.

Next, to endow the model with stronger exploratory abilities and expand its exploration space, we construct a prompt designed to encourage trial-and-error. As the yellow block in Fig. 2 shows,

the prompt contains three parts: task description, tool definition, and high-quality demos: a few manually crafted exemplars that demonstrate a general agentic reasoning process, including planning, interacting, and self-correction after errors. Then, we prompt strong models (e.g., Gemini 2.5 Flash [8] and DeepSeek-V3 [19]) to generate multiple candidate trajectories for each question.

After that, to ensure the correctness of the trajectories, we apply an outcome-based rejection sampling, filtering the generated trajectories using two criteria, retaining only those that are both correct and fully grounded in evidence:

- **Exact-match Correctness.** The Exact Match (EM) score [37] between the predicted and gold answers must be 1. This guarantees the correctness of the trajectory.
- **Evidence Grounding.** All predicted answers must appear in the observation history $\{o_0, \dots, o_T\}$, ensuring that the agent does not rely on internal parametric knowledge that could lead to hallucinations during training.

Finally, we filtered these trajectories to ensure that there are at most three trajectories for each question, and that the number of trajectories for each question category is balanced. Statistical information for the resulting cold start dataset is shown in Table 9 in the appendix D; this dataset is smaller than most existing methods.

4.2.2 Multi-Turn Finetuning. In the fine-tuning stage, we aim to teach the agent to reason over long, complex interaction histories. We use the entire history as input but compute the loss only on the agent’s generated tokens, masking out the observation part returned by tools during training. The process can be formalized as follows:

We define the input prompt as x and the expert trajectory as τ . Here the trajectory τ is represented as a sequence of M tokens,

$\tau = (y_1, \dots, y_M)$, which comprise the expert's thoughts and actions. The agent is fine-tuned by maximizing the likelihood of generating this expert trajectory conditioned on the input x . During training, the objective is formulated as:

$$\mathcal{L}_\tau(\theta) = - \sum_{j=1}^M \log P_\theta(y_j | x, y_{<j}), \quad (4)$$

where θ represents the model's parameters, M is the total number of tokens in the sequence, y_j is the j -th token, and $y_{<j}$ denotes the sequence of preceding tokens $(y_1, \dots, y_{j-1})$.

4.3 RL-based Exploration Stage

Based on the cold-start model, in the RL stage, we aim to further promote autonomous exploration and eliminate the reliance on process supervision. To achieve this, we employ the outcome-based Group Relative Policy Optimization (GRPO) method [29], where any trajectory leading to a correct answer receives a positive incentive, thereby encouraging the agent to explore and acquire stronger reasoning capabilities. Based on it, to ensure the RL training is both balanced and effective, we introduce a curriculum strategy on the reward, scheduling the reward strictness, guiding the learning process from easy to hard and effectively suppressing reward hacking.

4.3.1 RL Objective. We adopt the outcome-based reinforcement learning objective based on Group Relative Policy Optimization [29], which is defined as:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{\{ \tau^{(i)} \}_{i=1}^N \sim \pi_{\theta_{\text{old}}}(\cdot | q)} \left[\frac{1}{N} \sum_{i=1}^N \frac{1}{|\tau^{(i)}|} \sum_{j=1}^{|\tau^{(i)}|} \min \left(\rho_{i,j}(\theta) \hat{A}_{i,j}, \text{clip}(\rho_{i,j}(\theta), 1 \pm \epsilon) \hat{A}_{i,j} \right) \right] - \beta \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}), \quad (5)$$

$$\hat{A}_{i,j} = \frac{R(\tau^{(i)}) - \text{mean}(\{R(\tau^{(i)})\}_{i=1}^N)}{\text{std}(\{R(\tau^{(i)})\}_{i=1}^N)}, \quad (6)$$

$$\rho_{i,j}(\theta) = \frac{\pi_\theta(\tau_j^{(i)} | \tau_{<j}^{(i)})}{\pi_{\theta_{\text{old}}}(\tau_j^{(i)} | \tau_{<j}^{(i)})}. \quad (7)$$

Here, q is the question from the train set D_Q , and $\tau^{(i)}$ is the i -th trajectory generated by the policy model $\pi_{\theta_{\text{old}}}$ for q . $R(\tau^{(i)})$ is a scalar reward derived solely from the final outcome of the trajectory. $\hat{A}_{i,j}$ represents the group-wise relative advantage, which is estimated by normalizing the group-level rewards for the same query, normalized by the standard deviation $\text{std}(\cdot)$ to stabilize training. $\rho_{i,j}(\theta)$ denotes the token-level importance ratio. π_{ref} is a reference policy (i.e., the model after the cold-start stage), and β controls the strength of the KL loss $\mathbb{D}_{\text{KL}}(\cdot \| \cdot)$ towards the reference policy.

4.3.2 Reward and Curriculum strategy. During the earlier phases of RL training, the cold-start model often lack the capability to generate complete and correct reasoning trajectories for complex questions. This results in reward sparsity when relying solely on final outcomes, and meanwhile hinders effective exploration. Thus, we introduce a composite reward function that mitigates reward

sparsity by providing denser feedback signals, while guiding the exploration process through a carefully designed easy-to-hard curriculum. The reward is specifically composed of a Format Reward and a multi-phase Answer Reward.

Format Reward (R_{fmt}). To discourage invalid trajectories and ensure that answers can be reliably parsed for reward calculation, we introduce a format reward. This component checks whether a trajectory adheres to the required format, specifically the presence of the answer box `\boxed{\cdot}`. If the format is correct, we set $R_{\text{fmt}} = 0.1$; otherwise, $R_{\text{fmt}} = 0.0$.

Answer Reward (R_{ans}). To verify the correctness of the predicted answer and avoid a sparse reward signal, we evaluate the agreement between the predictions inside `\boxed{\cdot}` and the gold answers using F_β score. Compared with a simple EM score, the F_β score balances precision and recall via an adoptable hyperparameter β , which prevents agents from trivially obtaining a high score by returning a large, low-precision candidate set. Concretely, the F_β score is computed as:

$$\text{Precision} = \frac{|\hat{\mathcal{A}} \cap \mathcal{A}_q^*|}{|\hat{\mathcal{A}}|}, \quad \text{Recall} = \frac{|\hat{\mathcal{A}} \cap \mathcal{A}_q^*|}{|\mathcal{A}_q^*|}, \quad (8)$$

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}, \quad (9)$$

where $\hat{\mathcal{A}}$ denotes the de-duplicated set of answers produced by the model and $\mathcal{A}_q^*$ denotes the set of gold answers, and $|\cdot|$ denotes multiset cardinality.

Based on the answer reward, we employ a curriculum learning strategy by adapting the β parameter. This gradually adjusts the reward difficulty from easy to hard, as illustrated in the top-right part of Fig. 2.

- **Phase 1: Precision-focused Reward.** In early training, we set $\beta = 0.5$ in the answer reward. This precision-leaning objective grants relatively high reward for correct items even when recall is incomplete. As the Fig. 2 shows, if the golden answer is $\{A1, A2\}$, a precise but partial prediction $\{A1\}$ achieves a high reward ($r_2 = 0.83$). Conversely, an prediction that includes extraneous items, such as $\{A1, A2, A3, A4\}$, is penalized with a lower reward ($r_1 = 0.55$), encouraging accurate predictions and preventing the model from hacking the reward by emitting large candidate sets.
- **Phase 2: Balanced Reward.** Once the model achieves stable precision, we switch the objective to the F_1 score ($\beta = 1$), which balances precision and recall. Under this balanced metric, a partial prediction is less rewarded than in Phase 1, incentivizing the agent to explore more thoroughly and recover the full set of correct answers.

The total reward is the sum of the two parts, capped at a maximum value of 1.0:

$$R_{\text{total}} = \min(R_{\text{fmt}} + R_{\text{ans}}, 1.0). \quad (10)$$

5 Experiments

To prove the effectiveness of AGENTICKBQA, we conduct comprehensive experiments on widely used datasets. Specifically, we focus on the following research questions (RQ): **RQ1:** Does AGENTICKBQA outperform other directly comparable methods? **RQ2:** Does

Table 1: Experiment Results (in percentage) on WebQSP, CWQ, and GrailQA. The Bold and underlined numbers indicate the best and second-best low-resource performance.

Method	Backbone	WebQSP		CWQ		GrailQA			
		Overall F1	RHits@1	Overall F1	EM	I.I.D.	Compositional	Zero-shot	Overall
Prompting Methods									
IO	GPT-4-turbo	39.3	45.5	33.9	45.7	—	—	—	29.4
CoT	GPT-4-turbo	39.7	47.5	33.7	43.7	—	—	—	28.1
CoT+SC	GPT-4-turbo	39.0	47.1	36.6	47.5	—	—	—	29.6
KB-BINDER	Codex-davinci-002	52.6	—	—	—	43.3	36.6	44.0	42.2
KB-Coder	GPT-3.5-turbo	55.7	—	—	—	45.5	38.6	47.3	44.9
ARG-KBQA	GPT-3.5-turbo	58.8	—	—	—	51.5	41.8	52.1	48.5
Interactive-KBQA	GPT-4-turbo	71.2	72.5	49.1	59.2	—	—	—	—
Fine-tune-based Methods									
RnG-KBQA	BERT-base-uncased	75.6	—	—	—	89.0	68.9	74.7	76.9
TIARA	BERT-base-uncased	78.9	75.2	—	—	91.2	74.8	80.7	81.9
SFT-traj.	Llama2-7B	57.5	59.9	42.2	48.8	54.2	45.8	44.2	48.5
MCTS-KBQA	Llama-3.1-8B	<u>76.0</u>	76.2	<u>66.8</u>	<u>75.2</u>	—	—	—	—
KBQA-o1	Qwen2.5-7B	57.8	—	—	—	84.4	77.0	<u>75.7</u>	<u>77.9</u>
AGENTICKBQA-cold start	Qwen2.5-Coder-7B	56.1	66.3	54.8	59.8	60.6	63.5	67.2	63.1
AGENTICKBQA	Qwen2.5-Coder-7B	77.2	80.1	68.3	75.7	<u>81.1</u>	<u>70.6</u>	84.1	80.5

Table 2: Training instances (Train) and evaluation result with consumption (Evaluation), where Time represents the average number of rollouts required per question at inference, normalized to our method (1x).

Model	Train				Evaluation					
	P.S. #Inst.	O.S.#Inst.			CWQ		WebQSP		GrailQA	
		WebQSP	CWQ	GrailQA	Time	F1	Time	F1	Time	F1
MCTS-KBQA	750	2743	2843	—	3.24×	66.8	2.91×	76.0	—	—
KBQA-o1	240	2929	—	43851	—	—	6×	57.8	6×	77.9
Ours	0	2414	<u>4699</u>	3563	1×	68.3	1×	77.2	1×	80.5

the main components of AGENTICKBQA work effectively? **RQ3:** How does AGENTICKBQA progressively enhance the agent’s reasoning capability for the KBQA task with outcome-only supervision? **RQ4:** Does AGENTICKBQA generates flexible and robust reasoning trajectories?

5.1 Experiment Setup

Datasets and Metrics. We evaluate AGENTICKBQA on three widely-used KBQA benchmarks: WebQSP, CWQ, and the generalization-focused GrailQA. Following prior work [40], we primarily report the F1 score as the main metric, supplemented by Random Hits@1 (RHits@1) and Exact Match (EM) scores.

Baselines. We compare AGENTICKBQA against two primary categories of methods: *prompting-based* and *fine-tuning-based* methods. For the former, we include a diverse range of prompting strategies, ranging from direct prompting (e.g., IO and CoT) to agent-based reasoning methods such as Interactive-KBQA [40]. To ensure a fair comparison and reproducibility, we report the scores for these methods directly from their original publications. For the fine-tuning-based methods, we consider SOTA models fine-tuned on the full

dataset, including RnG-KBQA [43] and TIARA [32]. Furthermore, we evaluate directly comparable low-resource methods that are fine-tuned on limited data, such as the agentic approaches SFT-traj. [40], KBQA-o1 [23], and MCTS-KBQA [41]. Further details on the datasets, baselines, evaluation protocol, and implementation are provided in Appendix A.

5.2 Main Results - RQ1

The experiment results are presented in Table 1, and for additional context on the efficiency of our method, we compare the number of samples used during training with the number of rollouts used during inference in Table 2. Taken together, these results demonstrate that AGENTICKBQA establishes a new state-of-the-art for low-resource KBQA across all benchmarks, which can be analyzed from three key aspects.

First, AGENTICKBQA significantly outperforms all directly comparable low-resource baselines, particularly on the more complex CWQ and the generalization-focused GrailQA datasets. Meanwhile, it narrows the gap with fully-supervised methods while using only a fraction of the data (10k outcome-supervised samples vs. 60k fully-annotated queries), highlighting its exceptional data efficiency.

Second, results prove that the outcome-supervised multi-stage training strategy is effective than the process-supervised SFT strategy. Results in the last two blocks of Table 1 indicate that AGENTICKBQA consistently outperforms leading process-supervised SFT baselines. On GrailQA, AGENTICKBQA achieves an F1 score of 80.5%, which represents a 3.3% relative improvement over the prior state-of-the-art method KBQA-o1 while using 12× less training data (3.5k vs. 43.8k samples). While performance is slightly lower on GrailQA’s I.I.D. and compositional subsets, which favor extensive training data to cover specific patterns, our method demonstrates a relative

Table 3: Ablation Study on Three KBQA Datasets.

Model	GrailQA				WebQSP		CWQ	
	IID	Comp.	Zero-shot	Overall F1	F1	Hits@1	F1	EM
AGENTICKBQA	81.1	70.6	84.1	80.5	77.2	80.1	68.3	75.7
w/o reward curriculum								
EM as Reward	45.5	50.6	73.6	62.8	73.8	76.1	50.2	55.0
F1 as Reward	68.7	77.6	78.6	76.0	71.9	77.0	58.3	59.8
F0.5 as Reward	79.1	68.2	82.6	78.7	76.3	80.9	67.2	74.7
F1+F0.5 as Reward	74.9	65.5	82.6	76.1	73.8	77.0	64.6	66.1
w/o RL (cold-start)	60.6	63.5	67.2	63.1	56.7	64.9	50.5	59.8
w/o upsampling	52.5	53.0	62.3	60.2	56.1	66.3	54.8	52.7

improvement of up to 11.1% on the zero-shot subset. This proves its superior ability to reason effectively on truly unseen questions.

Furthermore, beyond higher overall scores, AGENTICKBQA is more efficient in both supervision and inference. In terms of supervision, as detailed in the “Train” columns of Table 2, AGENTICKBQA uses zero process-supervised (P.S.) instances and a comparable or smaller number of outcome-supervised (O.S.) samples than MCTS-based competitors. As mentioned before, AGENTICKBQA outperforms KBQA-o1 on GrailQA while using much fewer supervised samples. In terms of inference, as shown in the “Evaluation” columns, AGENTICKBQA employs a single linear reasoning pass and avoids the costly sampling required by MCTS, reducing latency by 3.2-6 times. These results support outcome supervision as a more effective and efficient alternative to incremental SFT on pre-defined or explored paths.

5.3 Ablation Study - RQ2

To verify the effectiveness of our key designs, we conduct a comprehensive ablation study, and the results are presented in Table 3. First, we assess the reward curriculum by replacing our proposed easy-to-hard ($F_{0.5}$ -then- F_1) strategy, give the result using EM, F_1 , $F_{0.5}$, and reversed strategy F_1 -then- $F_{0.5}$ as reward, respectively. Next, we evaluate the contribution of the entire RL stage by removing it altogether. Finally, we remove the data sampling strategy from the initial cold-start phase. The results confirm that each component is integral to the model’s performance, as removing or modifying any of them leads to a noticeable degradation.

Effectiveness of the Easy-to-Hard Reward Curriculum. The first block of Table 3 shows the effectiveness of the Easy-to-Hard Reward Curriculum. Take the GrailQA dataset as an example, we observe that an overly simplistic signal like EM is insufficient for exploration; it induces over-recall reward hacking, resulting in a massive drop in the F_1 score on GrailQA. Meanwhile, a stricter F_1 reward is too punishing early on, leading to a -4.5% F_1 drop. Notably, reversing the curriculum to a hard-to-easy schedule (F_1 -then- $F_{0.5}$) also degrades performance. This proves that starting with an easy task encourages broad exploration to discover correct reasoning paths, while the subsequent hard task refines the agent’s ability to achieve a better balance between precision and recall.

Critical Contribution of the RL Stage. From the second block of Table 3, we can observe that removing the entire RL-stage causes at least 17.4% absolute drop in the F_1 score. This large gap suggests that RL-based exploration is essential for refining the agent’s reasoning capability beyond initial supervised learning.

Importance of Upsampling in the Cold-Start Stage. Finally, as the last row shows, removing the up-sampling of infrequent question types leads to poor performance on those specific categories. This underscores the importance of our sampling strategy for building a model that can generalize across a diverse range of reasoning patterns.

5.4 Analysis - RQ3

To answer the RQ3 and investigate how AGENTICKBQA progressively enhances the agent’s reasoning capability under the outcome-only supervision, we analyze from the train dynamics, the evolution of its reasoning robustness and diversity, and the comparative performance against process-supervised methods.

5.4.1 Analysis of Training Dynamics with Outcome-Only Supervision. As shown in Fig. 3, to track the evolution of the model’s capabilities during training, we present the curves of (a) training reward, (b) response length (token), (c) turns of interaction, and (d) the number of invalid tool calls along the training step. It can be seen that the agent’s strategy clearly transitions from **broad exploration** to **efficient exploitation**: initially, the agent explores inefficiently, which is reflected in high response lengths and turn counts for a low reward. As training progresses, these trends reverse: the reward consistently rises while the metrics for response length, turns, and invalid tool calls all significantly decrease. This dynamic illustrates a fundamental improvement in the training process, not just a superficial optimization of rewards.

5.4.2 Analysis of Reasoning Capability. To investigate whether RL enhances the agent’s reasoning beyond mere SFT imitation, we analyze its behavior across two dimensions. Fig. 4(a) illustrates its robustness by showing the composition of rollout trajectories, categorized as: “Directly Hit,” for trajectories where the final prediction hits at least one gold answers; “Recovered after Error/Empty,” for those that hit at least one gold answers after encountering an intermediate error or empty observation; and “Incorrect,” for trajectories where the final prediction fails to find any gold answers. It shows that the proportion of successful trajectories (“Directly Hit” and “Recovered”) steadily increases during training. This confirms that the agent learns to effectively recover from imperfect feedback, a direct advantage of our outcome supervision approach. In contrast, process supervision encourages the model to simply replicate pre-defined trajectories from the training data, and thus fails to incentivize this type of recovery behavior.

Fig. 4(b) measures flexibility by plotting the diversity of trajectories, which we define as the number of unique final SPARQL queries generated for the same question across the rollouts. It can be seen that the agent initially broadens its search for a wide range of solutions before consolidating on a compact set of the most reliable ones, which is consistent with the process discussed in Sec. 5.4.1. This indicates that outcome-only supervision naturally fosters an exploration-then-convergence behavior, where the agent ultimately retains sufficient solution diversity to effectively address the lack of flexibility inherent in process-supervised methods.

5.4.3 Analysis of Process Reward. To explore whether incorporating process supervision in addition to outcome-only supervision benefits model performance, we experimented by incorporating

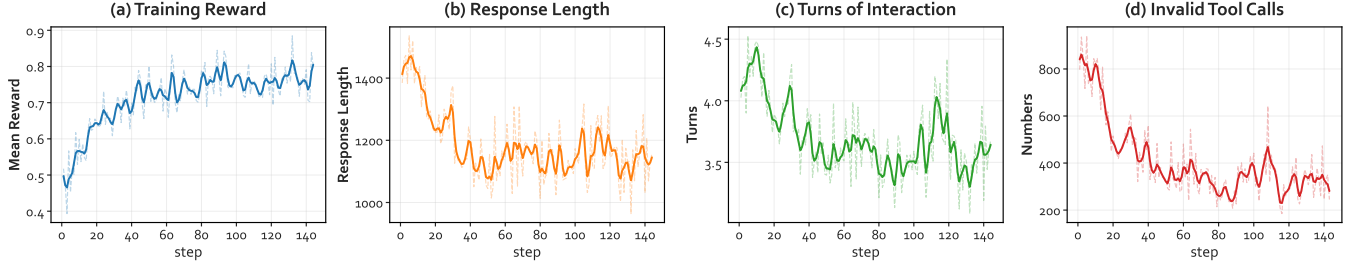


Figure 3: Training curves for AGENTICKBQA, illustrating: (a) training reward, (b) response length, (c) interaction turns, and (d) the number of invalid tool calls.

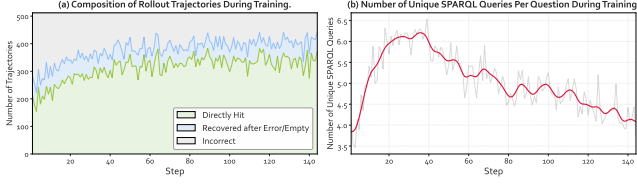


Figure 4: Evolution of Robustness and Flexibility during training: (a) Robustness, shown by the composition of rollout trajectories, and (b) Flexibility, shown by the number of unique SPARQL queries per question.

necessary process-level penalties for behaviors including hallucinations and timeouts. Specifically, we define a hallucination as a tool call containing a relation or type that is not present in the observation history, while a timeout query denotes a call that results in a 300 second timeout. We applied a negative reward of -0.2 for each occurrence of these behaviors, with the total accumulated penalty capped at -0.5. We then compared our full model against two variants incorporating these Process Rewards (P.R.): one applying the penalties throughout the entire RL training (P.R.@RL-phase-1&2), and another applying them only during the second stage (P.R.@RL-phase-2). The results clearly demonstrate that introducing penalties for any intermediate behaviors, even for intuitively negative ones, degrades final performance. We hypothesize that in KBQA tasks, where the correctness of answers can be explicitly verified, process-level rewards may interfere with the agent’s capacity to form a robust trial-and-error exploration strategy, as they tend to prematurely penalize exploratory trajectories that could eventually achieve correct results.

Table 4: Performance comparison of different models and training stages on the WebQSP, CWQ, and GrailQA.

Model	WebQSP		CWQ		GrailQA	
	F1	Hit@1	F1	EM	F1	EM
P.R.@RL-phase-1&2	71.5	75.3	64.9	72.7	73.0	76.2
P.R.@RL-phase-2	73.3	78.2	68.0	74.8	78.1	81.0
AGENTICKBQA	77.2	80.1	68.3	75.7	80.5	83.7

5.5 Case Study - RQ4

To concretely demonstrate the advanced reasoning capabilities of AGENTICKBQA, we present a qualitative case study with two detailed examples in Appendix B. These examples highlight the agent’s ability to: (1) robustly recover from erroneous feedback; (2) flexibly use tools to find correct answers via reasoning trajectories that differ from the gold SPARQL. These cases serve as strong evidence of the robustness and flexibility fostered by the proposed AGENTICKBQA approach. In addition, we conduct error analysis, which is detailed in Appendix E.

6 Conclusion

In this paper, we focused on a key limitation of existing agentic KBQA paradigms: their reliance on process supervision, which provides weak incentives for autonomous exploration. To overcome this, we proposed AGENTICKBQA, our first agentic reasoning model for KBQA. It is trained by a multi-stage curriculum reinforcement learning framework that effectively leverages outcome-only supervision through a carefully designed learning curriculum. Specifically, AGENTICKBQA first establishes foundational reasoning capabilities via a cold-start fine-tuning stage on a small, high-quality dataset created through outcome-based rejection sampling. It then enhances the agent’s exploratory capacity through curriculum RL, employing a progressive reward schedule that transitions from easy to hard tasks. Extensive experiments demonstrate the effectiveness of our approach. Further analyses reveal that AGENTICKBQA learns to act as a robust and flexible agent, capable of recovering from errors and strategically exploring diverse reasoning trajectories. Future work may investigate more advanced reflection mechanisms to mitigate remaining error types and extend this curriculum strategy to other complex, agent-based reasoning tasks.

Acknowledgments

This work is supported grants from NSFC No. 62372430, the Youth Innovation Promotion Association CAS No.2023112, the National Key Research and Development Program of China No.2024YFC3308200 and the National Natural Science Foundation of China No.62306299. We also thank the anonymous reviewers for their valuable comments and suggestions.

References

- [1] Antoine Bordes, Nicolas Usunier, Sumit Chopra, and Jason Weston. 2015. Large-scale simple question answering with memory networks. *arXiv preprint*

- arXiv:1506.02075 (2015).
- [2] Fangshu Chen, Yilin Huang, Jiahui Wang, Chengcheng Yu, and Xiankai Meng. 2025. Ifqa-llm: intelligent intention-driven financial question-answering with large language models. *The Journal of Supercomputing* 81, 13 (2025), 1–32.
 - [3] Yu Chen, Lingfei Wu, and Mohammed J Zaki. 2019. Bidirectional Attentive Memory Networks for Question Answering over Knowledge Bases. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2913–2923.
 - [4] Zhuo Chen, Zhao Zhang, Zixuan Li, Fei Wang, Yutao Zeng, Xiaolong Jin, and Yongjun Xu. 2024. Self-Improvement Programming for Temporal Knowledge Graph Question Answering. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 14579–14594.
 - [5] Zi-Yuan Chen, Chih-Hung Chang, Yi-Pei Chen, Jijnasa Nayak, and Lun-Wei Ku. 2019. UHop: An Unrestricted-Hop Relation Extraction Framework for Knowledge-Based Question Answering. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 345–356.
 - [6] Mingyue Cheng, Jie Ouyang, Shuo Yu, Ruiran Yan, Yucong Luo, Zirui Liu, Daoyu Wang, Qi Liu, and Enhong Chen. 2025. Agent-r1: Training powerful llm agents with end-to-end reinforcement learning. *arXiv preprint arXiv:2511.14460* (2025).
 - [7] Sitao Cheng, Ziyuan Zhuang, Yong Xu, Fangkai Yang, Chaoyun Zhang, Xiaoting Qin, Xiang Huang, Ling Chen, Qingwei Lin, Dongmei Zhang, et al. 2024. Call Me When Necessary: LLMs can Efficiently and Faithfully Reason over Structured Environments. In *Findings of the Association for Computational Linguistics ACL 2024*. 4275–4295.
 - [8] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multi-modality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
 - [9] Haishuo Fang, Xiaodan Zhu, and Iryna Gurevych. 2024. DARA: Decomposition-Alignment-Reasoning Autonomous Language Agent for Question Answering over Knowledge Graphs. In *Findings of the Association for Computational Linguistics ACL 2024*. 3406–3432.
 - [10] Guangze Gao, Zixuan Li, Chunfeng Yuan, Jiawei Li, Wu Jianzhuo, Yuehao Zhang, Xiaolong Jin, Bing Li, and Weiming Hu. 2025. D-RAG: Differentiable Retrieval-Augmented Generation for Knowledge Graph Question Answering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 35386–35405.
 - [11] Yu Gu, Xiang Deng, and Yu Su. 2023. Don't Generate, Discriminate: A Proposal for Grounding Language Models to Real-World Environments. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4928–4949.
 - [12] Yu Gu, Sue Kase, Michelle Vanni, Brian Sadler, Percy Liang, Xifeng Yan, and Yu Su. 2021. Beyond iid: three levels of generalization for question answering on knowledge bases. In *Proceedings of the web conference 2021*. 3477–3488.
 - [13] Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. 2024. QueryAgent: A Reliable and Efficient Reasoning Framework with Environmental Feedback based Self-Correction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5014–5035.
 - [14] Heewon Jang, Yeongtaek Oh, Seunghee Jin, Haemin Jung, Hyesoo Kong, Dokyung Lee, Dongkyu Jeon, and Wooju Kim. 2017. KBQA: Constructing structured query graph from keyword query for semantic search. In *Proceedings of the International Conference on Electronic Commerce*. 1–8.
 - [15] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2025. Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9505–9523.
 - [16] Jinhao Jiang, Kun Zhou, Xin Zhao, and Ji-Rong Wen. [n.d.]. UniKGQA: Unified Retrieval and Reasoning for Solving Multi-hop Question Answering Over Knowledge Graph. In *The Eleventh International Conference on Learning Representations*.
 - [17] Yunshi LAN and Jing JIANG. [n.d.]. Query graph generation for answering multi-hop complex questions from knowledge bases.(2020). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 969–974.
 - [18] Tianle Li, Xueguang Ma, Alex Zhuang, Yu Gu, Yu Su, and Wenhui Chen. 2023. Few-shot In-context Learning on Knowledge Base Question Answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6966–6980.
 - [19] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
 - [20] Biyang Liu, Huimin Yu, and Guodong Qi. 2022. Graftnet: Towards domain generalized stereo matching with a broad-spectrum and task-oriented feature. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13012–13021.
 - [21] Guangyi Liu, Yongqi Zhang, Yong Li, and Quanming Yao. [n.d.]. Dual Reasoning: A GNN-LLM Collaborative Framework for Knowledge Graph Question Answering. In *The Second Conference on Parsimony and Learning (Proceedings Track)*.
 - [22] Yantao Liu, Zixuan Li, Xiaolong Jin, Yucan Guo, Long Bai, Saiping Guan, Jiafeng Guo, and Xueqi Cheng. 2024. An in-context schema understanding method for knowledge base question answering. In *International Conference on Knowledge Science, Engineering and Management*. Springer, 419–434.
 - [23] Haoran Luo, E Haihong, Yikai Guo, Qika Lin, Xiaobao Wu, Xinyu Mu, Wenhao Liu, Meina Song, Yifan Zhu, and Anh Tuan Luu. [n.d.]. KBQA-o1: Agentic Knowledge Base Question Answering with Monte Carlo Tree Search. In *Forty-second International Conference on Machine Learning*.
 - [24] Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston. 2016. Key-Value Memory Networks for Directly Reading Documents. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.
 - [25] Zhijie Nie, Richong Zhang, Zhongyuan Wang, and Xudong Liu. 2024. Code-style in-context learning for knowledge-based question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18833–18841.
 - [26] Yunqi Qiu, Yuanzhuo Wang, Xiaolong Jin, and Kun Zhang. 2020. Stepwise reasoning for multi-relation question answering over knowledge graph with weak supervision. In *Proceedings of the 13th international conference on web search and data mining*. 474–482.
 - [27] Hongyu Ren, Hanjun Dai, Bo Dai, Xinyun Chen, Michihiro Yasunaga, Haitian Sun, Dale Schuurmans, Jure Leskovec, and Denny Zhou. 2021. Lego: Latent execution-guided reasoning for multi-hop question answering on knowledge graphs. In *International conference on machine learning*. PMLR, 8959–8970.
 - [28] Apoorv Saxena, Aditay Tripathi, and Partha Talukdar. 2020. Improving multi-hop question answering over knowledge graphs using knowledge base embeddings. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 4498–4507.
 - [29] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
 - [30] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv:2409.19256* (2024).
 - [31] Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. 2021. TransferNet: An Effective and Transparent Framework for Multi-hop Question Answering over Relation Graph. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 4149–4158.
 - [32] Yiheng Shu, Zhiwei Yu, Yuhua Li, Börje Karlsson, Tingting Ma, Yuzhong Qu, and Chin-Yew Lin. 2022. TIARA: Multi-grained Retrieval for Robust Question Answering over Large Knowledge Base. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 8108–8121.
 - [33] Miao Su, Zixuan Li, Zhuo Chen, Long Bai, Xiaolong Jin, and Jiafeng Guo. 2024. Temporal knowledge graph question answering: A survey. *arXiv preprint arXiv:2406.14191* (2024).
 - [34] Haitian Sun, Tania Bedrax-Weiss, and William Cohen. 2019. PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2380–2390.
 - [35] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. [n.d.]. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *The Twelfth International Conference on Learning Representations*.
 - [36] Yawei Sun, Lingling Zhang, Gong Cheng, and Yuzhong Qu. 2020. SPARQA: skeleton-based semantic parsing for complex questions over knowledge bases. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 8952–8959.
 - [37] Alon Talmor and Jonathan Berant. 2018. The Web as a Knowledge-Base for Answering Complex Questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 641–651.
 - [38] Yuhang Tian, Dandan Song, Zhijiang Wu, Changzhi Zhou, Hao Wang, Jun Yang, Jing Xu, Ruanmin Cao, and Haoyu Wang. 2024. Augmenting Reasoning Capabilities of LLMs with Graph Structures in Knowledge Base Question Answering. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 11967–11977.
 - [39] Junde Wu, Jiayuan Zhu, Yunli Qi, Jingkun Chen, Min Xu, Filippo Menolascina, and Vicente Grau. 2024. Medical graph rag: Towards safe medical large language model via graph retrieval-augmented generation. *arXiv preprint arXiv:2408.04187* (2024).
 - [40] Guanming Xiong, Junwei Bao, and Wen Zhao. 2024. Interactive-KBQA: Multi-Turn Interactions for Knowledge Base Question Answering with Large Language

- Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10561–10582.
- [41] Guanming Xiong, Haochen Li, and Wen Zhao. 2025. MCTS-KBQA: Monte Carlo Tree Search for Knowledge Base Question Answering. *arXiv preprint arXiv:2502.13428* (2025).
 - [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
 - [43] Xi Ye, Semih Yavuz, Kazuma Hashimoto, Yingbo Zhou, and Caiming Xiong. 2022. RNG-KBQA: Generation Augmented Iterative Ranking for Knowledge Base Question Answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6032–6043.
 - [44] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. 2022. Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5773–5784.
 - [45] Ruilin Zhao, Feng Zhao, Long Wang, Xianzhi Wang, and Guandong Xu. 2024. KG-CoT: chain-of-thought prompting of large language models over knowledge graphs for knowledge-aware question answering. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*. 6642–6650.
 - [46] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. LlamaFactory: Unified Efficient Fine-Tuning of 100+ Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, Bangkok, Thailand. <http://arxiv.org/abs/2403.13372>
 - [47] Yuqi Zhu, Shuofei Qiao, Yixin Ou, Shumin Deng, Shiwei Lyu, Yue Shen, Lei Liang, Jinjie Gu, Huajun Chen, and Ningyu Zhang. 2025. KnowAgent: Knowledge-Augmented Planning for LLM-Based Agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 3709–3732.

A Experiment Setup

A.1 Datasets

We evaluate the reasoning performance on three widely-used datasets: WebQSP [37], CWQ [37], and GrailQA [12]. These datasets are constructed based on Freebase and consist of natural language questions paired with their corresponding SPARQL queries. Specifically, WebQSP features questions where the topic entity and answer entities are at most 2 relation hops apart. CWQ extends WebQSP by incorporating more complex entities and constraints, with relation paths up to 4 hops. In contrast to WebQSP and CWQ, GrailQA is designed for a more comprehensive evaluation of generalization capabilities across three settings: independently and identically distributed (I.I.D.), compositional, and zero-shot. We follow the setting of previous work [40], which uniformly sample questions across different types in WebQSP and CWQ. For the GrailQA dataset, we randomly sample 600 instances.

A.2 Baselines

Our experiments compare against two categories of methods.

Fine-tuning-based Methods. We consider two settings: fine-tuning on partial data (low-source setting) and on full data. In the low-source setting, we select the agentic approaches, includes the SFT model in Interactive-KBQA [40], which uses the same toolset as our method (abbreviated as SFT-traj.), two MCTS-based agentic methods, KBQA-o1 [23] and MCTS-KBQA [41]. Notably, since our method is trained on only a subset of the data and uses no SPARQL annotations, its performance is directly comparable to this category of baselines. For the full-data setting, which serves as a secondary point of reference, we include the SOTA SP-based methods RnG-KBQA [43] and TIARA [32].

Prompting-based Methods. We selected a diverse set of prompting strategies for comparison. These include methods based on direct prompting, Chain-of-Thought (CoT), and self-consistency with CoT; approaches that use an LLM to enhance semantic parsing

(KB-BINDER [18] and KB-Coder [25]) and subgraph retrieval [38]; and Interactive-KBQA [40] that prompts an LLM to reason as an agent.

A.3 Implementation Details

We adopt Qwen2.5-Coder-7B-Base as the backbone LLM for all experiments. For the cold-start phase, we fine-tune the model using the LoRA algorithm via the LlamaFactory framework [46]. We employ a cosine learning rate schedule with a peak learning rate of $2e-5$, a batch size of 8, and a maximum sequence length of 4096. The model is fine-tuned for 8 epochs to ensure a stable initialization of the agentic reasoning capabilities. The RL training is implemented using the Agent-R1 framework [6], which is built upon the verl [30]. We utilize the GRPO objective and train for 1 epoch. The training configuration includes a global batch size of 512, with a rollout batch size of 64 and $N = 8$ rollouts per prompt. The learning rate is fixed at $1e-6$ without warmup. We set the maximum prompt length to 4096 and the response length to 8192 tokens, with a KL loss coefficient of 0.001. During training, the rollout temperature is set to 1.0 to encourage exploration, while a temperature of 0.0 is used during validation to ensure deterministic outputs.

B Case Study

In this section, we present two representative case studies. The first case demonstrates robustness: when faced with two consecutive empty query results, the agent successfully recovers by leveraging a more distant observation from its history to try a new, promising relation, ultimately finding the correct answer. The second case highlights flexibility and self-correction, showing the agent’s ability to identify an illogical query result (e.g., receiving a high school when a university was expected) and adapt its strategy by employing a novel action to resolve the issue.

C Further Analysis

In this section, we provide several supplementary analyses, including the impact of different backbone models on performance, the efficiency of our method and a further analysis of the joint distribution of error frequency and sample accuracy during the training.

C.1 Impact of Different Backbones

We compared the impact of different backbone models on performance during the cold-start stage. As shown in Table 7, our results indicate that Qwen2.5-Coder-7B-Base delivers the strongest overall performance. Although Qwen3-8B-Base performs better on the simpler WebQSP dataset, we selected Qwen2.5-Coder-7-Base for its superior performance on CWQ, which contains more complex questions. Furthermore, our task involves numerous code-style tool calls, where a coder-specialized model has a natural advantage.

C.2 Efficiency

To prove the efficiency of our method, we conducted a computational cost analysis against prompting-based baselines, including the single-round inference method CoT and the multi-round interactive method ToG. As shown in Table 8, the comparison covers LLM call frequency, token usage, and overall cost. The results reveal that AGENTICKBQA achieves performance comparable to the

Table 5: Example of robust reasoning of AGENTICKBQA.

Question: who plays london tipton in suite life on deck?	
THINK	I need to find the actor who plays London Tipton in "The Suite Life on Deck". I will start by searching for predicates related to "actor" or "performance" for the entity "London Tipton(m.07g8r3)".
TOOL CALL	SearchGraphPatterns(sparql="SELECT ?e WHERE { VALUES ?e {ns:m.07g8r3} }", semantic="actor/performer")
RESPONSE	[..., (?e, film.film_character.portrayed_in_films -> film.performance.actor, "Brenda Song"),...]
THINK	...I will use the film.film_character.portrayed_in_films -> film.performance.actor predicate to find the actor who played London Tipton in "The Suite Life on Deck"
TOOL CALL	ExecuteSPARQL(sparql="SELECT ?actor WHERE { VALUES ?e1 {ns:m.07g8r3} . VALUES ?e2 {ns:m.03mj4jm} . ?e1 ns:film.film_character.portrayed_in_films ?cvt . ?cvt ns:film.performance.actor ?actor . ?cvt ns:film.performance.film ?e2 . }")
RESPONSE	[]
THINK	The previous query returned no results. This might be due to the structure of the predicates or the way the entities are linked. I will try a different approach by searching for predicates related to "actor" or "performance"...
TOOL CALL	SearchGraphPatterns(sparql="SELECT ?e WHERE { VALUES ?e {ns:m.03mj4jm} }", semantic="actor/performer/character")
RESPONSE	[..., (?e, award.award_nominated_work.award_nominations -> award.award_nomination.award, "Kids' Choice Award for Favorite TV Actor"), ...]
THINK	...award.award_nominated_work.award_nominations -> award.award_nomination.award_nominee, This predicate seems to link a show to actors...
TOOL CALL	ExecuteSPARQL(sparql="SELECT ?actor WHERE { VALUES ?e1 {ns:m.07g8r3} . VALUES ?e2 {ns:m.03mj4jm} . ?e2 ns:award.award_nominated_work.award_nominations ?cvt . ?cvt ns:award.award_nomination.award_nominee ?actor . ?cvt ns:award.award_nomination.character ?e1 . }")
RESPONSE	[]
THINK	The previous queries have not yielded the correct result. I need to refine my approach. The question asks for the actor who plays London Tipton in The Suite Life on Deck. I will try a different strategy by focusing on the relationship between the character and the film...I will try to find the actor who played London Tipton in any film and then filter for "The Suite Life on Deck".
TOOL CALL	ExecuteSPARQL(sparql="SELECT ?actor WHERE { VALUES ?e {ns:m.07g8r3} . ?e ns:film.film_character.portrayed_in_films ?cvt . ?cvt ns:film.performance.actor ?actor . }")
RESPONSE	["Brenda Song"]
ANSWER	Brenda Song

Table 6: Example of flexible reasoning of AGENTICKBQA.

Question: Where did Charles Drew attend college that has the latest founding date?	
THINK	I need to find the colleges that Charles R. Drew attended and then find the college with the latest founding date.
TOOL CALL	SearchGraphPatterns(sparql="SELECT ?e WHERE { VALUES ?e {ns:m.018t67} }", semantic="attended college")
(Omit for space...)	
TOOL CALL	ExecuteSPARQL(sparql="SELECT DISTINCT ?college WHERE { VALUES ?e {ns:m.018t67} . ?e ns:people.person.education ?edu . ?edu ns:education.education.institution ?college . ?college ns:organization.organization.date_founded ?founded_date . } ORDER BY DESC(xsd:date(?founded_date)) LIMIT 1")
RESPONSE	["Dunbar High School"]
THINK	The result is "Dunbar High School", but the question asks for the college, not high school. I need to re-evaluate the search. ... I will search for the type "College/University".
TOOL CALL	SearchTypes(query="College/University")
RESPONSE	["education.university", ...]
THINK	The "SearchTypes" results include "education.university" ... I will find the colleges attended by Charles R. Drew and their founding dates, then order by founding date in descending order and limit to 1 to get the college with the latest founding date.
TOOL CALL	ExecuteSPARQL(sparql="SELECT DISTINCT ?college WHERE { VALUES ?e {ns:m.018t67} . ?e ns:people.person.education ?edu . ?edu ns:education.education.institution ?college . ?college ns:type.object.type ns:education.university . ?college ns:organization.organization.date_founded ?founded_date . } ORDER BY DESC(xsd:date(?founded_date)) LIMIT 1")
RESPONSE	["McGill University Faculty of Medicine"]
ANSWER	McGill University Faculty of Medicine

Table 7: Performance comparison of various backbone models across three benchmark datasets.

Model	WebQSP		CWQ		GrailQA	
	F1	Hits@1	F1	EM	F1	EM
Qwen2.5-Coder-3B-Base	58.4	63.9	36.7	41.2	58.4	68.2
Qwen2.5-Coder-7B-Base	56.7	64.9	50.5	59.8	63.1	68.4
Qwen3-4B-Base	63.9	69.4	38.4	39.0	55.0	59.1
Qwen3-8B-Base	65.3	71.6	44.6	45.8	60.1	63.1

complex ToG, while maintaining a computational budget similar to the much simpler CoT. This efficiency stems from outcome-based supervision: the model learns to recover from empty results or intermediate errors in a single pass, obviating the need to generate multiple branches to find a solution at significant computational expense. Overall, AGENTICKBQA shifts the efficiency frontier toward lower cost while maintaining strong effectiveness.

Table 8: Cost and effectiveness on CWQ and WebQSP.

CWQ				
Model	# Avg. LLM Call	Total Token	Total Cost	Hit@1
CoT	1.0	409.7	8.00×10^{-5}	43.7
ToG	9.2	11468.5	2.30×10^{-3}	69.5
Ours	<u>5.0</u>	<u>660.6</u>	0	75.7

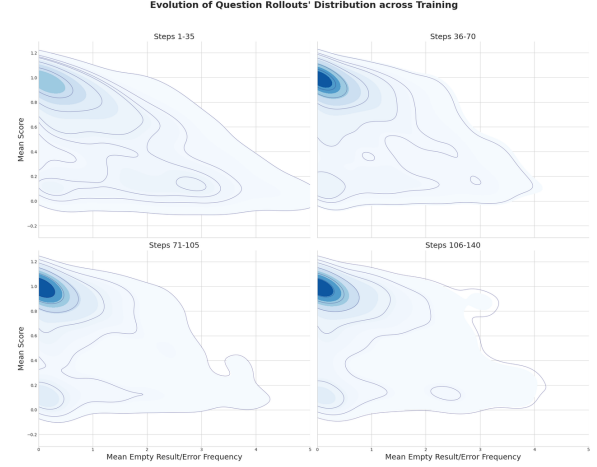
WebQSP				
Model	# Avg. LLM Call	Total Token	Total Cost	Hit@1
CoT	1.0	397.6	8.00×10^{-5}	47.5
ToG	8.8	10189.4	2.10×10^{-3}	82.6
Ours	<u>3.9</u>	<u>439.6</u>	0	<u>80.1</u>

C.3 Overall Robustness Capability Shift

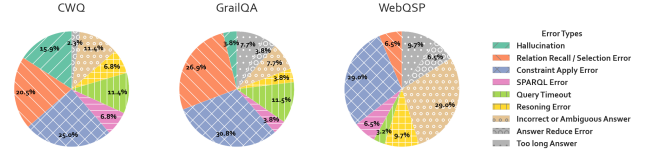
We analyze the association between intermediate errors and final rewards throughout training. As illustrated in Fig. 5, we plot the joint distribution of the mean frequency of empty results/errors per question against the mean reward. Early in training, the distribution forms a downward-sloping triangle, where higher error rates are strongly correlated with lower rewards. As training advances, this shape flattens into a rectangle, indicating that high rewards are achieved even with a non-trivial frequency of errors. Concurrently, the density increases in the upper-left corner, reflecting a growing number of questions being solved directly and perfectly. The transformation provides compelling evidence of our method’s superiority: the agent not only learns to avoid mistakes but also develops the crucial ability to recover from them, transitioning from a brittle execution policy to a robust, high-scoring one.

D Statistics

We give the statistic of training samples used in cold-start-stage and RL stage on three datasets.

**Figure 5: Joint distribution of mean empty-result/error frequency and mean reward across training phases.****Table 9: Number of samples for different training stages across the WebQSP, CWQ, and GrailQA datasets.**

Stage	WebQSP	CWQ	GrailQA
RL-Phase-1	1265	2400	1800
RL-Phase-2	709	1600	1200
Cold-Start	440	699	563

**Figure 6: Analysis of sampled failure cases per dataset. We visualize the proportion of factors contributing to errors.**

E Error Analysis

To identify the deficiency of AGENTICKBQA, we conducted an error analysis by manually inspecting random sampled cases where the F_1 score was less than 1. The results are categorized in Fig. 6, with the primary error types being Hallucination, Constraint Application Error, and Relation Recall/Selection Error. Hallucinations often result from flawed self-correction attempts. While intuitively detrimental, our prior experiments show that penalizing such behavior actually stifles the model’s capacity for self-correction. The other two error types generally stem from incorrect decisions during complex reasoning. These errors highlight the space for future work: enhancing the model’s ability to handle complex questions within a linear decision-making process. Notably, the high rate of constraint errors on WebQSP is due to the model failing to identify implicit temporal constraints (e.g., the KB’s 2015 cutoff date) in present-tense questions.